

Product Forge Multi-Agent Pipeline — Complete Workflow Reference

Version: 2.0 (Redesign) **Date:** 2026-09-01 **Purpose:** Single source of truth for building pipelines that produce working software

Table of Contents

1. [Executive Summary](#)
2. [Pipeline Stages \(16 Stages\)](#)
3. [Orchestrator Design](#)
4. [Agent Design](#)
5. [Implementation Phasing Strategy](#)
6. [State Management](#)
7. [Human-in-the-Loop Design](#)

8. [Knowledge Base Integration](#)

9. [Prompt Templates](#)

10. [Verification Strategy](#)

1. Executive Summary

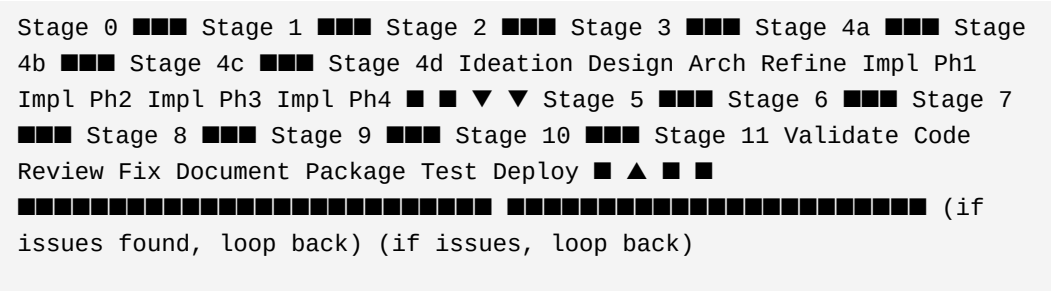
What This Pipeline Does

This pipeline takes a human idea and produces working, deployable software through a series of agent-driven stages. Each stage produces verifiable output, and a human approves before the pipeline advances.

Critical Problems with the Current Pipeline

Problem	Impact	Root Cause
Agents produce reports, not working code	Product never runs	No verification that code actually executes
Scaffolding passes as implementation	13 features are stubs	No intermediate verification between features

Stage Overview



Stage 0: Ideation

Field	Value
Agent	orchestrator
Input	User's idea (free text)
Output	products/<project>/docs/product-plan.md, products/<project>/pipeline.json
Verification	Human reviews scope and confirms
Human Gate	YES — Must approve scope before Stage 1
Critical Checks	Product plan includes: project name, vision, 10-15 features, priorities, MVP scope, success criteria

product-plan.md must contain: - Project name and one-line vision - 10-15 user stories / features (each with acceptance criteria) - Priority labels: P0 (must have), P1 (should have), P2 (nice to have) - MVP scope (which features are P0) - Success criteria (how we know it works) - Target users - Out-of-scope items (only with user approval)

pipeline.json initialization:

```
{ "name": "<project-name>", "current_stage": "0", "pipeline_complete": false, "stages": { "0": { "name": "Ideation", "status": "in_progress", "agent": "orchestrator", "human_approved": false } } }
```

Stage 1: Design

Field	Value
Agent	design
Input	docs/product-plan.md (MUST read in full)
Output	docs/requirements.md, docs/design.md

Verification	Human reviews requirements count matches product-plan.md
Human Gate	YES — Must approve before Stage 2
Critical Checks	Every feature in product-plan.md has a requirement; no features dropped without approval

requirements.md must contain: - Functional Requirements (FR-001 through FR-015+, one per feature) - Each FR has: ID, description, acceptance criteria, priority, dependencies - Non-Functional Requirements section (22+ subsections) - Traceability matrix: which FR maps to which feature in product-plan.md

design.md must contain: - UI/UX wireframes (ASCII art or description) - Component hierarchy - Page layouts - Navigation flow - Color scheme, typography - Responsive breakpoints - Accessibility requirements (WCAG 2.1 AA)

Stage 2: Architecture

Field	Value
Agent	architect
Input	docs/requirements.md, docs/design.md (MUST read in full)
Output	docs/architecture.md, docs/architecture-spec.json
Verification	Human reviews architecture + all 22 NFR subsections
Human Gate	YES — Must approve before Stage 3
Critical Checks	All 22+ NFR subsections addressed; tech stack decisions justified; ADRs written

architecture.md must contain: - System overview diagram (ASCII art) - Tech stack (with version numbers) - Component architecture - Database schema - API design (REST endpoints) - Authentication flow - File structure - All 22+ NFR subsections (Performance, Security, Accessibility, Scalability, etc.) - Architecture Decision Records (ADRs) for major choices

architecture-spec.json must contain:

```
{ "tech_stack": { "backend": "Python 3.11 + FastAPI", "database": "PostgreSQL 15", "frontend": "React 18 + TypeScript", "mobile": "React Native", "infrastructure": "Docker" }, "features": { "F-001": { "name": "Authentication", "components": ["auth-api", "auth-ui", "auth-db"], "endpoints": ["POST /auth/login", "POST /auth/register"], "priority": "P0" } }, "nfrs": { "performance": { ... }, "security": { ... }, "accessibility": { ... } } }
```

Stage 3: Refine Requirements

Field	Value
Agent	orchestrator (with user)
Input	docs/architecture.md (review for feasibility)
Output	Updated docs/requirements.md (if changes needed), docs/pipeline-state.md
Verification	Human confirms architecture aligns with requirements
Human Gate	YES — Must approve before implementation
Critical Checks	No phantom requirements (features with no implementation path); all FRs have clear implementation guidance

Purpose: This stage catches mismatches between what was designed and what is architecturally feasible. The orchestrator reviews the architecture against requirements and flags any gaps.

Stage 4a: Implement Phase 1 (Foundation + Core)

Field	Value
Agent	implement
Input	docs/architecture.md, docs/requirements.md (FRs for Phase 1 only), docs/architecture-spec.json
Output	Working code for features: Authentication, Dashboard, Search, ToDo

Verification	docker compose build succeeds; human opens localhost:3000 and tests each feature
Human Gate	YES — Human must test and approve each feature in browser
Critical Checks	Auth flow works end-to-end; Dashboard shows data; Search returns results; ToDo CRUD works

Features in Phase 1: | Feature | FR IDs | What Must Work | |-----|-----|-----|
 | Authentication | FR-001 | Register, login, logout, session persistence | | Dashboard |
 FR-002 | Shows user data, widgets load, responsive layout | | Search | FR-003 | Search
 across entities, results display, filters work | | ToDo | FR-004 | Create, read, update,
 delete, mark complete, filter |

Verification checklist after Phase 1:

```

■ docker compose build – exits 0 ■ docker compose up -d – all
services start ■ Open localhost:3000 – page loads ■ Register new
account – works ■ Login with credentials – works ■ Dashboard shows
widgets – works ■ Search returns results – works ■ Create ToDo item –
works ■ Complete ToDo item – works ■ Delete ToDo item – works ■ Human
approves in browser
  
```

Stage 4b: Implement Phase 2 (Content Features)

Field	Value
Agent	implement
Input	docs/architecture.md, docs/requirements.md (FRs for Phase 2), Phase 1 code
Output	Working code for: Calendar, Goals, News, Health + Phase 1 still works
Verification	All Phase 1 features still work; Phase 2 features work; docker build succeeds
Human Gate	YES — Human tests all features

Critical Checks	No regression in Phase 1; Phase 2 features are functional
-----------------	---

Features in Phase 2: | Feature | FR IDs | What Must Work | |-----|-----|-----|
| Calendar | FR-005 | Event creation, view, edit, delete, date navigation | | Goals |
FR-006 | Goal creation, progress tracking, milestones | | News | FR-007 | Feed display,
article view, save/bookmark | | Health | FR-008 | Health metrics, tracking, visualization |

Stage 4c: Implement Phase 3 (Specialized Features)

Field	Value
Agent	implement
Input	docs/architecture.md, docs/requirements.md (FRs for Phase 3), Phase 1+2 code
Output	Working code for: Exploratory, Spiritual, Documents, Financial + Phase 1+2 still work
Verification	All previous features still work; Phase 3 features work
Human Gate	YES
Critical Checks	No regression; complex features actually function

Features in Phase 3: | Feature | FR IDs | What Must Work | |-----|-----|-----|
| Exploratory | FR-009 | Content exploration, recommendations, navigation | | Spiritual |
FR-010 | Spiritual content, journaling, reflections | | Documents | FR-011 | Document
upload, view, organize, search | | Financial | FR-012 | Financial tracking, budgets,
reports |

Stage 4d: Implement Phase 4 (Mobile)

Field	Value
Agent	implement (mobile specialization)
Input	All web code, docs/architecture.md, docs/design.md (mobile section)
Output	React Native mobile app with all 13 screens

Verification	Mobile app builds; runs on device/emulator; all screens navigate correctly
Human Gate	YES — Human tests on phone
Critical Checks	All screens render; navigation works; API calls succeed; responsive to device sizes

Stage 5: Validate

Field	Value
Agent	validate
Input	All code files, test files
Output	docs/reports/issues.md
Verification	MUST actually run pytest and report real pass/fail counts
Human Gate	NO — Auto-routes to Fix if failures found
Critical Checks	Reports ACTUAL pass/fail counts, not "tests exist"; identifies real bugs

Validation runs:

```
1. pytest --tb=short --cov=app --cov-report=term-missing → Record: X
passed, Y failed, Z skipped → Record: coverage percentage
2. docker compose build → Record: success or failure
3. docker compose up -d → Record: services start or fail
4. curl localhost:3000 → Record: 200 OK or error
```

issues.md format:

```
# Validation Report Date: 2026-09-01 Pipeline Stage: 5 (Validate) ##
Test Results - Total tests: 47 - Passed: 42 - Failed: 5 - Skipped: 0 -
Coverage: 73% ## Failed Tests 1. test_auth_login_returns_token -
AssertionError (line 42) 2. test_todo_create_empty_title -
TimeoutError (line 87) ... ## Build Results - docker compose build:
SUCCESS - docker compose up: SUCCESS - localhost:3000 responds: YES
(200 OK) ## Recommendation - 5 tests failing → Route to Fix agent
```

Stage 6: Code Review

Field	Value
Agent	code-review
Input	ALL code files + docs/requirements.md, docs/feature-status.md
Output	docs/reports/code-review.md
Verification	Must check for TODOs, stubs, empty functions, mock data
Human Gate	NO — Auto-routes to Fix if issues found
Critical Checks	CAN reject implementation if scaffolding/stubs found

Code review checks:

■ No TODO comments in code ■ No stub functions (pass, ..., raise
 NotImplementedError) ■ No mock/hardcoded data in production code ■ No
 empty catch blocks ■ No console.log/print statements left in ■ All
 functions have implementations ■ All API endpoints return real data ■
 All database queries execute real SQL ■ No commented-out code blocks >
 10 lines ■ Error handling exists for all external calls

code-review.md format:

```

# Code Review Report Date: 2026-09-01 Pipeline Stage: 6 (Code Review)
## Summary - Files reviewed: 47 - Issues found: 3 (1 critical, 2
minor) ## Critical Issues 1. app/services/todo_service.py:45 -
function `get_todos` returns hardcoded list → REJECT: Must return
real database query ## Minor Issues 1. app/auth/login.py:12 - print
statement left in 2. app/models/user.py:23 - unused import ##
Recommendation - 1 critical issue → Route to Fix agent

```

Stage 7: Fix

Field	Value
Agent	fix
Input	docs/reports/issues.md, docs/reports/code-review.md
Output	Fixed code
Verification	Re-run pytest after fix; re-run code review checks

Human Gate	NO — Returns to Validate after fix
Critical Checks	All previously failing tests now pass; all code review issues resolved

Fix agent workflow:

```
1. Read issues.md and code-review.md
2. For each issue:
  a. Locate the problematic code
  b. Implement the fix
  c. Verify the fix doesn't break other tests
3. Run pytest – all tests must pass
4. Run code review checks – all must pass
5. Update issues.md with "FIXED" status for each issue
6. Return to Validate stage
```

Stage 8: Document

Field	Value
Agent	document
Input	docs/architecture.md, docs/requirements.md, all code
Output	README.md, API documentation, user guides
Verification	Human reads docs and confirms they make sense
Human Gate	YES — Human must approve documentation
Critical Checks	README has setup instructions that actually work; API docs match endpoints

README.md must contain: - Project description - Prerequisites - Setup instructions (copy-pasteable) - Running instructions - API overview - Contributing guidelines

Stage 9: Package

Field	Value
Agent	package
Input	All built code, docs/architecture.md

Output	Dockerfile, docker-compose.yml, CI/CD config
Verification	MUST actually run docker compose build and verify it succeeds
Human Gate	YES — Human sees Docker build output
Critical Checks	Build exits 0; images are < 500MB; services start correctly

Package verification:

```
1. docker compose build → Must exit 0 → Record build time and image sizes
2. docker compose up -d → All services must start → Record: which services are healthy
3. docker compose ps → All services must show "Up" status
4. curl localhost:3000 → Must return 200 OK
```

Stage 10: Test (Full Suite)

Field	Value
Agent	validate + NFR agents
Input	All code, all test files
Output	docs/reports/performance-report.md, docs/reports/security-audit-report.md, docs/reports/a11y-audit-report.md
Verification	MUST actually run k6, OWASP ZAP, axe-core — not just check files exist
Human Gate	YES — Human reviews all reports before deploy
Critical Checks	Performance meets NFR targets; no critical security vulnerabilities; WCAG 2.1 AA compliance

Test runs:

```
Performance: k6 run --out json=results.json load-test.js → Record: p50, p95, p99 latencies → Compare against NFR targets
Security: owasp-zap-cli quick-scan http://localhost:3000 → Record: High, Medium, Low findings → Zero High/Medium findings required
```

Accessibility: axe-core http://localhost:3000 → Record: violations by severity → Zero serious/critical violations required

Stage 11: Deploy

Field	Value
Agent	deploy
Input	Docker images, CI/CD config
Output	Deployment to staging, monitoring setup
Verification	MUST actually build and deploy to staging
Human Gate	YES — Must approve before production
Critical Checks	Staging environment works; monitoring alerts configured

Deploy verification:

1. Deploy to staging environment 2. Verify staging URL responds 3. Run smoke tests against staging 4. Configure monitoring alerts 5. Human reviews staging 6. Human approves production deploy 7. Deploy to production 8. Verify production URL responds

3. Orchestrator Design

Current Problems

Problem	Impact	Fix
Orchestrator has <code>edit:deny</code>	Can't update <code>pipeline.json</code>	Grant <code>edit:allow</code> for <code>pipeline.json</code> only
Auto-advances without human approval	Broken output propagates	Add mandatory human gate after every agent


```
pipeline.json ■ Update agent-audit.md ■ Update pipeline-state.md ■ ■  
■■■■■■■■
```

Orchestrator Prompt Template

```
You are the Orchestrator for [PROJECT_NAME]. Your job is to run each  
stage of the pipeline, present output to the human, and wait for  
approval. RULES (NON-NEGOTIABLE): 1. NEVER skip stages – always run  
them in order 2. NEVER advance without explicit human approval 3.  
ALWAYS show the human what was created (list files, show key content)  
4. ALWAYS ask: "Approve this stage and continue?" 5. ONLY advance if  
human says "approve", "yes", "go", or similar 6. If human says "fix  
this", "change X", "re-do Y" → send back to the agent with the  
feedback 7. UPDATE pipeline.json after every stage transition 8.  
VERIFY output files exist and are non-empty before presenting to human  
9. If an agent produces empty or invalid output, REJECT it and ask  
agent to redo 10. Maintain an audit trail in agent-audit.md CURRENT  
STATE: - Project: [PROJECT_NAME] - Current Stage: [STAGE_NUM]  
([STAGE_NAME]) - Pipeline: pipeline.json STAGE [N] – [STAGE_NAME]: -  
Agent: [AGENT_NAME] - Input files: [INPUT_FILES] - Expected output:  
[OUTPUT_FILES] After the agent completes, present the output to the  
human with: 1. What files were created 2. Summary of what was done 3.  
How to verify (command to run, URL to open, etc.) 4. "Approve this  
stage and continue?"
```

4. Agent Design

4.1 Design Agent

Field	Value
Role	Convert product vision into detailed requirements and UI/UX design
Input	docs/product-plan.md (MUST read in full)
Output	docs/requirements.md, docs/design.md
Constraints	NO scope reduction without user approval; every feature in product-plan.md must appear in requirements
Verification	Count features in product-plan.md vs requirements.md — must match
Chunking	N/A (single task)

System Prompt:

You are the Design Agent for [PROJECT_NAME]. READ FIRST: - products/[project]/docs/product-plan.md (read completely) YOUR JOB: 1. Create products/[project]/docs/requirements.md with: - Functional Requirements (FR-001 through FR-0XX) - Each FR: ID, description, acceptance criteria, priority, dependencies - Non-Functional Requirements (22+ subsections) - Traceability matrix 2. Create products/[project]/docs/design.md with: - UI/UX wireframes (ASCII art) - Component hierarchy - Page layouts - Navigation flow - Color scheme, typography - Responsive breakpoints - Accessibility requirements (WCAG 2.1 AA) RULES: - EVERY feature from product-plan.md must have a corresponding FR - Do NOT mark features as "out of scope" without explicit user approval - Do NOT reduce scope – the user defined the scope, you detail it - Write acceptance criteria that are testable (can be verified) - Write NFRs with measurable targets (e.g., "page loads in < 2 seconds")

Verification checklist:

- requirements.md exists and is non-empty
- design.md exists and is non-empty
- Number of FRs >= number of features in product-plan.md
- Each FR has: ID, description, acceptance criteria, priority
- NFR section has 22+ subsections
- Traceability matrix maps FRs to features
- design.md includes accessibility section
- Human approves

4.2 Architect Agent

Field	Value
Role	Design system architecture and technical decisions
Input	docs/requirements.md, docs/design.md (MUST read in full)
Output	docs/architecture.md, docs/architecture-spec.json
Constraints	Must address ALL 22+ NFR subsections; no decisions without justification
Verification	Human reviews architecture + all NFR subsections are present
Chunking	N/A (single task)

System Prompt:

You are the Architect Agent for [PROJECT_NAME]. READ FIRST: - products/[project]/docs/requirements.md (read completely) - products/[project]/docs/design.md (read completely) YOUR JOB: 1. Create products/[project]/docs/architecture.md with: - System overview diagram (ASCII art) - Tech stack (with version numbers) - Component architecture - Database schema (SQL CREATE statements) - API design (REST endpoints with request/response) - Authentication flow - File structure (directory tree) - ALL 22+ NFR subsections (Performance, Security, Accessibility, etc.) - Architecture Decision Records (ADRs) 2. Create products/[project]/docs/architecture-spec.json with: - Tech stack details - Feature-to-component mapping - Endpoint specifications - NFR targets RULES: - Every FR must have a clear implementation path in the architecture - Every NFR must have a measurable target - Every major decision must have an ADR - Do NOT skip NFR subsections - Tech stack must be justified (why these technologies?) - Database schema must include all entities from requirements

Verification checklist:

■ architecture.md exists and is non-empty ■ architecture-spec.json exists and is valid JSON ■ System diagram present ■ Tech stack with versions listed ■ Database schema covers all entities ■ API endpoints cover all FRs ■ 22+ NFR subsections present ■ ADRs for major decisions ■ Human approves

4.3 Implement Agent (Split Into Phases)

THIS IS THE CRITICAL CHANGE.

Field	Value
Role	Write working code for features
Input	docs/architecture.md, docs/requirements.md (relevant FRs only), docs/architecture-spec.json
Output	Working code for the assigned phase's features
Constraints	Must produce RUNNABLE code, not stubs; must not break previous phases
Verification	docker compose build succeeds; human tests in browser
Chunking	Split into 4 phases (see below)

Phase Structure

PHASE 1: Foundation + Core Features (Stage 4a) ■■■■ Features: Authentication, Dashboard, Search, ToDo ■■■■ Why: Most-used, most critical features ■■■■ Input: architecture.md, requirements.md (FR-001 through FR-004) ■■■■ Output: Working code for 4 features ■■■■ Verification: docker build + human tests in browser ■■■■ Human Gate: YES ■■■■ Dependency: Stages 0-3 complete PHASE 2: Content Features (Stage 4b) ■■■■ Features: Calendar, Goals, News, Health ■■■■ Why: Content-heavy, can be added incrementally ■■■■ Input: architecture.md, requirements.md (FR-005 through FR-008) + Phase 1 code ■■■■ Output: Working code for 4 features + Phase 1 still works ■■■■ Verification: All previous features + new features work ■■■■ Human Gate: YES ■■■■ Dependency: Phase 1 complete and working PHASE 3: Specialized Features (Stage 4c) ■■■■ Features: Exploratory, Spiritual, Documents, Financial ■■■■ Why: Complex features that depend on foundation ■■■■ Input: architecture.md, requirements.md (FR-009 through FR-012) + Phase 1+2 code ■■■■ Output: Working code for 4 features + Phase 1+2 still work ■■■■ Verification: All previous features + new features work ■■■■ Human Gate: YES ■■■■ Dependency: Phase 2 complete PHASE 4: Mobile (Stage 4d) ■■■■ Features: Mobile app (all 13 screens) ■■■■ Why: Mobile depends on web being complete and API stable ■■■■ Input: All web code, architecture.md, design.md (mobile section) ■■■■ Output: React Native mobile app ■■■■ Verification: Builds, runs on device, all screens work ■■■■ Human Gate: YES ■■■■ Dependency: Phase 3 complete

Implement Agent System Prompt

You are the Implement Agent for [PROJECT_NAME], Phase [N]. READ FIRST:
 - products/[project]/docs/architecture.md (read completely) -
 products/[project]/docs/requirements.md (read completely, but
 implement only Phase [N] FRs) -
 products/[project]/docs/architecture-spec.json (read completely) YOUR
 JOB: Write working, runnable code for Phase [N] features:
 [FEATURE_LIST] RULES: 1. DO NOT write stubs, TODOs, or placeholder
 code 2. DO NOT use mock data – connect to real database 3. DO NOT skip
 error handling 4. Every function must have a real implementation 5.
 Every API endpoint must return real data 6. Every database query must
 execute real SQL 7. Write tests for each feature 8. Ensure docker
 compose build succeeds 9. If this is Phase 2+, ensure Phase 1 features
 still work 10. Update feature-status.md after each feature OUTPUT: -
 Code files in app/ directory - Test files in tests/ directory -
 Updated feature-status.md - Report of what was implemented

4.4 Code Review Agent

Field	Value
Role	Review all code for quality, completeness, and standards compliance
Input	ALL code files + docs/requirements.md, docs/feature-status.md

Output	docs/reports/code-review.md
Constraints	Must check for TODOs, stubs, empty functions, mock data; CAN reject implementation
Verification	Report includes specific file:line references for every issue
Human Gate	NO (auto-routes to Fix if critical issues found)

System Prompt:

```
You are the Code Review Agent for [PROJECT_NAME]. READ FIRST: - All files in products/[project]/app/ (read completely) - products/[project]/docs/requirements.md - products/[project]/docs/feature-status.md YOUR JOB: Review every code file for: 1. TODO comments 2. Stub functions (pass, ..., raise NotImplementedError) 3. Mock/hardcoded data in production code 4. Empty catch blocks 5. Console.log/print statements left in 6. Unimplemented functions 7. Missing error handling 8. Security issues 9. Performance issues 10. Accessibility issues OUTPUT: - products/[project]/docs/reports/code-review.md - Format: file:line – severity – description - CRITICAL: Can reject implementation if scaffolding is found RULES: - Be thorough – check EVERY file - Report SPECIFIC locations (file:line) - Classify as CRITICAL or MINOR - CRITICAL = code is a stub, returns mock data, or is not implemented - MINOR = style issue, unused import, etc.
```

Verification checklist:

```
■ code-review.md exists and is non-empty ■ Every code file was reviewed ■ All TODOs identified ■ All stubs identified ■ All mock data identified ■ All issues have file:line references ■ Severity classification for each issue ■ Recommendation: PASS or FAIL (with reasons)
```

4.5 Validate Agent

Field	Value
Role	Run tests and report actual results
Input	All code files, all test files
Output	docs/reports/issues.md
Constraints	MUST actually run pytest; MUST report real pass/fail counts

Verification	Output contains actual pytest output, not just "tests exist"
Human Gate	NO (auto-routes to Fix if failures found)

System Prompt:

```
You are the Validate Agent for [PROJECT_NAME]. YOUR JOB: 1. Run actual tests and report real results STEPS: 1. Run: pytest --tb=short --cov=app --cov-report=term-missing - Record: total tests, passed, failed, skipped - Record: coverage percentage - Record: full output of failed tests 2. Run: docker compose build - Record: success or failure - Record: build time 3. Run: docker compose up -d - Record: services started 4. Run: curl -s -o /dev/null -w "%{http_code}" http://localhost:3000 - Record: HTTP status code OUTPUT: - products/[project]/docs/reports/issues.md - Format: - Test results (actual numbers) - Failed tests (with error messages) - Build results - Recommendation RULES: - NEVER claim "PASS" without running the actual command - NEVER report "tests exist" as a result – report "X passed, Y failed" - Include ACTUAL pytest output in the report - Include ACTUAL error messages for failed tests
```

Verification checklist:

```
■ issues.md exists and is non-empty ■ Contains actual pytest output (not just summary) ■ Contains actual pass/fail counts ■ Contains actual error messages for failures ■ Contains build results ■ Contains HTTP status check ■ Recommendation is based on actual results
```

4.6 NFR Agents

Performance Agent

Field	Value
Role	Verify performance requirements are met
Input	docs/architecture.md (NFR section) + code
Output	docs/reports/performance-report.md
Constraints	MUST actually run k6, not just check files exist
Verification	Report contains actual latency numbers
Human Gate	YES

Security Agent

Field	Value
Role	Verify security requirements are met
Input	docs/architecture.md (NFR section) + code
Output	docs/reports/security-audit-report.md
Constraints	MUST actually run OWASP ZAP, not just check files exist
Verification	Report contains actual vulnerability findings
Human Gate	YES

Accessibility Agent

Field	Value
Role	Verify accessibility requirements are met
Input	docs/architecture.md (NFR section) + code
Output	docs/reports/a11y-audit-report.md
Constraints	MUST actually run axe-core, not just check files exist
Verification	Report contains actual violation counts
Human Gate	YES

4.7 Fix Agent

Field	Value
Role	Fix issues identified by Validate and Code Review agents
Input	docs/reports/issues.md, docs/reports/code-review.md
Output	Fixed code
Constraints	Cannot advance until all tests pass
Verification	Re-run pytest after fix

Human Gate	NO (returns to Validate)
------------	--------------------------

System Prompt:

```
You are the Fix Agent for [PROJECT_NAME]. READ FIRST: -
products/[project]/docs/reports/issues.md (list of issues to fix) -
products/[project]/docs/reports/code-review.md (code review findings)
YOUR JOB: For each issue listed in the reports: 1. Locate the
problematic code 2. Understand why it's failing 3. Implement the fix
4. Verify the fix doesn't break other tests STEPS: 1. For each FAILED
test in issues.md: a. Read the test file b. Read the implementation
file c. Fix the implementation d. Re-run that specific test: pytest
tests/test_x.py::test_name -v e. Verify it passes 2. For each CRITICAL
issue in code-review.md: a. Read the code file at the specified line
b. Fix the issue (replace stub, implement function, etc.) c. Verify
the fix is correct 3. After all fixes: a. Run: pytest --tb=short b.
ALL tests must pass c. If any fail, fix them too OUTPUT: - Updated
code files - Updated issues.md (mark issues as FIXED) - Report of what
was fixed
```

4.8 Package Agent

Field	Value
Role	Create Docker and CI/CD configuration
Input	All built code, docs/architecture.md
Output	Dockerfile, docker-compose.yml, CI/CD config
Constraints	MUST actually run docker compose build and verify it succeeds
Verification	Build exits 0; services start; human sees Docker build output
Human Gate	YES

Verification checklist:

```
■ Dockerfile exists and is valid ■ docker-compose.yml exists and is
valid ■ docker compose build exits 0 ■ docker compose up -d starts
all services ■ docker compose ps shows all services "Up" ■ curl
localhost:3000 returns 200 OK ■ Human approves Docker build output
```

4.9 Document Agent

Field	Value
-------	-------

Role	Create user-facing documentation
Input	docs/architecture.md, docs/requirements.md, all code
Output	README.md, API docs, user guides
Constraints	Setup instructions must actually work
Verification	Human reads docs and confirms they make sense
Human Gate	YES

4.10 Deploy Agent

Field	Value
Role	Deploy to staging/production
Input	Docker images, CI/CD config
Output	Deployment to staging, monitoring setup
Constraints	MUST actually build and deploy to staging; human must approve production
Verification	Staging works; monitoring alerts configured
Human Gate	YES — must approve before production

5. Implementation Phasing Strategy

Why Phase Implementation

The current pipeline tries to implement ALL 13 features at once. This fails because:

■ PROBLEM 1: Too Much Context ■ ■

■ ■ A single agent run trying to implement 13 features exceeds ■ ■ context limits. The agent starts strong but degrades, producing ■ ■ scaffolding for later features. ■

■ PROBLEM 2: No Intermediate Verification ■ ■

■ ■ Without checkpoints, bugs compound. Feature 1 has 3 bugs, ■ ■ Feature 2 has 4 bugs, ... Feature 13 has 7 bugs. By the end, ■ ■ the product has 50+ bugs with no clear fix path. ■

■ PROBLEM 3: Scaffolding Is Easier Than Implementation ■ ■

■ ■ When tired or context-limited, the agent writes stubs instead ■ ■ of real code. Without verification, stubs pass as "complete". ■

■ PROBLEM 4: No Human Feedback Until the End ■ ■

■ ■ The human doesn't see the product until Stage 11. By then, ■ ■ fixing fundamental design issues requires rewriting everything. ■

Phase Structure

PHASE 1: Foundation + Core Features ■■■ Duration: 1 agent run (focused on 4 features) ■■■ Features: Authentication, Dashboard, Search, ToDo ■■■ Why: These are the most-used, most critical features ■■■ Verification: docker build + human tests each feature in browser ■■■ Dependency: Stages 0-3 complete ■■■ Output: Working web app with 4 features

PHASE 2: Content Features ■■■ Duration: 1 agent run ■■■ Features: Calendar, Goals, News, Health ■■■ Why: Content-heavy features that can be added incrementally ■■■ Verification: All Phase 1 features still work + Phase 2 features work ■■■ Dependency: Phase 1 complete and working ■■■ Output: Working web app with 8 features

PHASE 3: Specialized Features ■■■ Duration: 1 agent run ■■■ Features: Exploratory, Spiritual, Documents, Financial ■■■ Why: Complex features that depend on foundation ■■■ Verification: All Phase 1+2 features still work + Phase 3 features work ■■■ Dependency: Phase 2 complete ■■■ Output: Working web app with 12 features

PHASE 4: Mobile ■■■ Duration: 1 agent run ■■■ Features: Mobile app (all 13 screens) ■■■ Why: Mobile depends on web being complete and API stable ■■■ Verification: Mobile app builds, runs, all screens work ■■■ Dependency: Phase 3 complete ■■■ Output: Working mobile app

Verification After Each Phase

After each implementation phase, the system MUST:

■ VERIFICATION SEQUENCE (after every phase) ■ ■

■ ■ ■ ■ Step 1: docker compose build ■ ■ → Must exit 0 ■ ■ → If fails → Fix agent → re-build ■ ■ ■ ■ Step 2: docker compose up -d ■ ■ → All services must start ■ ■ → If fails → Fix agent → re-start ■ ■ ■ ■ Step 3: Show the human the URL ■ ■ → "Open http://localhost:3000 in your browser" ■ ■ ■ ■ Step 4: Human opens browser and tests the features ■ ■ → Human clicks through each feature ■ ■ → Human creates data, edits, deletes ■ ■ → Human verifies the feature works ■ ■ ■ ■ Step 5: Human says "approve" or

"list issues" ■ ■ → Approve → advance to next phase ■ ■ → Issues → Fix agent → re-test → re-approve ■ ■ ■ ■ Step 6: Only after human approval → next phase ■



6. State Management

Files Updated by Each Agent

File	Updated By	When	Purpose
pipeline.json	orchestrator	Every stage transition	Pipeline state machine
docs/agent-context.md	each agent	After completing work	Resume state if interrupted
docs/pipeline-state.md	each agent	After completing work	Human-readable stage progression
docs/agent-audit.md	each agent	After completing work	Audit trail (what happened, when)
docs/feature-status.md	implement agent	After each feature	Feature tracking (which features work)
docs/reports/*.md	validate/review agents	After each validation	Findings and recommendations

pipeline.json Schema

```
{ "name": "myworld", "current_stage": "4a", "pipeline_complete": false, "started_at": "2026-09-01T10:00:00Z", "updated_at": "2026-09-01T14:30:00Z", "stages": { "0": { "name": "Ideation", "status": "completed", "agent": "orchestrator", "timestamp": "2026-09-01T10:00:00Z", "human_approved": true, "output_files": [ "products/myworld/docs/product-plan.md" ] }, "1": { "name": "Design", "status": "completed", "agent": "design", "timestamp": "2026-09-01T10:15:00Z", "human_approved": true, "output_files": [ "products/myworld/docs/requirements.md", "products/myworld/docs/design.md" ] }, "2": { "name": "Architecture", "status": "completed", "agent": "architect", "timestamp":
```

```
"2026-09-01T10:30:00Z", "human_approved": true, "output_files": [
"products/myworld/docs/architecture.md",
"products/myworld/docs/architecture-spec.json" ] }, "3": { "name":
"Refine Requirements", "status": "completed", "agent": "orchestrator",
"timestamp": "2026-09-01T10:45:00Z", "human_approved": true,
"output_files": [ "products/myworld/docs/requirements.md" ] }, "4a": {
"name": "Implement Phase 1", "status": "in_progress", "agent":
"implement", "features": ["F-001", "F-002", "F-003", "F-004"],
"human_approved": false, "output_files": [] }, "4b": { "name":
"Implement Phase 2", "status": "pending", "agent": "implement",
"features": ["F-005", "F-006", "F-007", "F-008"], "human_approved":
false, "output_files": [] }, "4c": { "name": "Implement Phase 3",
"status": "pending", "agent": "implement", "features": ["F-009",
"F-010", "F-011", "F-012"], "human_approved": false, "output_files":
[] }, "4d": { "name": "Implement Phase 4", "status": "pending",
"agent": "implement", "features": ["F-013"], "human_approved": false,
"output_files": [] }, "5": { "name": "Validate", "status": "pending",
"agent": "validate", "human_approved": false, "output_files": [] },
"6": { "name": "Code Review", "status": "pending", "agent":
"code-review", "human_approved": false, "output_files": [] }, "7": {
"name": "Fix", "status": "pending", "agent": "fix", "human_approved":
false, "output_files": [] }, "8": { "name": "Document", "status":
"pending", "agent": "document", "human_approved": false,
"output_files": [] }, "9": { "name": "Package", "status": "pending",
"agent": "package", "human_approved": false, "output_files": [] },
"10": { "name": "Test", "status": "pending", "agent": "validate+nfr",
"human_approved": false, "output_files": [] }, "11": { "name":
"Deploy", "status": "pending", "agent": "deploy", "human_approved":
false, "output_files": [] } } }
```

State File Update Rules

Every agent **MUST** update these files after completing work:

1. agent-context.md — Resume state for interrupted runs

```
# Agent Context Last updated: [TIMESTAMP] Agent: [AGENT_NAME] Stage:
[STAGE_NUM] ([STAGE_NAME]) Status: [completed/in_progress/failed] ##
What was done - [list of actions taken] ## What's next - [list of
pending actions] ## Resume instructions If resuming, start by:
[specific instruction]
```

2. pipeline-state.md — Human-readable progression

```
# Pipeline State Last updated: [TIMESTAMP] Project: [PROJECT_NAME]
Current Stage: [STAGE_NUM] ([STAGE_NAME]) ## Completed Stages - [x]
Stage 0: Ideation – completed [TIMESTAMP] - [x] Stage 1: Design –
completed [TIMESTAMP] - [ ] Stage 2: Architecture – in_progress ##
Pending Stages - Stage 3: Refine Requirements - Stage 4a: Implement
Phase 1 - ...
```

3. agent-audit.md — Audit trail

Agent Audit Trail Project: [PROJECT_NAME] ## [TIMESTAMP] –
[AGENT_NAME] – Stage [STAGE_NUM] Action: [what was done] Input files:
[files read] Output files: [files created] Duration: [how long]
Result: [success/failure] Notes: [any important notes]

7. Human-in-the-Loop Design

Why Human-in-the-Loop is Essential

■ WITHOUT Human-in-the-Loop: ■ ■

■ ■ Agent writes stubs → Orchestrator accepts → Pipeline advances ■
■ → Next agent builds on stubs → More stubs → Pipeline "completes" ■
■ → Human sees product for first time → Nothing works ■ ■ ■ ■ WITH
Human-in-the-Loop: ■ ■

■ ■ Agent writes code → Orchestrator presents to human ■ ■ → Human
tests in browser → "This doesn't work" ■ ■ → Agent fixes → Human
tests again → "Approve" ■ ■ → Pipeline advances with verified,
working code ■

How Human-in-the-Loop Works

Agent completes work ■ ▼ Orchestrator presents output to human ■ ■ ■ ■
What files were created ■ ■ ■ ■ What the agent claims was done ■ ■ ■ ■ How
to verify (URL, command, etc.) ■ ▼ Human reviews output ■ ■ ■ ■ Can
say "approve" to advance ■ ■ ■ ■ Can say "fix [specific issue]" to send
back ■ ■ ■ ■ Can say "show me more" to get details ■ ▼ If approve →
advance to next stage If fix → send back to agent, then re-present

Required Human Gates

After Stage	What Human Reviews	How to Review	Time Estimate
0 Ideation	Scope, features, priorities	Read product-plan.md	5 min
1 Design	Requirements, UX design	Read requirements.md + design.md	10 min
2 Architecture	Tech stack, NFRs, ADRs	Read architecture.md	10 min

3 Refine	Architecture-feasibility alignment	Read updated requirements.md	5 min
4a Impl Phase 1	Working auth + dashboard + search + todos	Open localhost:3000, test features	15 min
4b Impl Phase 2	Working calendar + goals + news + health	Open localhost:3000, test features	15 min
4c Impl Phase 3	Working exploratory + spiritual + docs + financial	Open localhost:3000, test features	15 min
4d Impl Phase 4	Working mobile app	Test on phone	15 min
8 Document	User docs	Read README	5 min
9 Package	Docker build succeeds	Run docker compose build, see output	5 min
10 Test	Performance, security, accessibility reports	Read reports, ask questions	10 min
11 Deploy	Staging works	Open staging URL, test features	10 min

Human Response Format

The human should respond with one of: - approve / yes / go / continue → Advance to next stage - fix [description] → Send back to agent with the issue - show me [detail] → Agent provides more information - reject [reason] → Stage must be redone entirely

8. Knowledge Base Integration

Current Problem

22 engineering guidelines exist in docs/guidelines/ but none of the agents reference them. They were created but never used.

Fix: Agent-Guideline Mapping

Each agent contract must include which guidelines to read:

Agent	Read These Guidelines
Design	coding/typescript/style-guide.md, frontend/react.md, ui-ux/accessibility.md
Architect	architecture/decisions.md, cloud/aws.md, infrastructure/docker.md
Implement (Web)	coding/python/style-guide.md, backend/fastapi.md, database/postgresql.md, api/rest.md
Implement (Mobile)	coding/typescript/style-guide.md, frontend/react-native.md, mobile/design-system.md
Code Review	security/owasp.md, testing/standards.md
Validate	testing/standards.md, monitoring/observability.md
Security NFR	security/owasp.md, security/authentication.md
Performance NFR	performance/standards.md, monitoring/observability.md
Accessibility NFR	ui-ux/accessibility.md, ui-ux/wcag.md
Package	infrastructure/docker.md, infrastructure/ci-cd.md
Deploy	infrastructure/docker.md, infrastructure/aws.md, monitoring/observability.md

How Agents Load Guidelines

Each agent's system prompt includes:

```
BEFORE writing any code, read these guidelines: -  
products/[project]/docs/guidelines/[path].md FOLLOW these guidelines  
in your output: - [summarize key rules from the guideline] If a  
guideline conflicts with the architecture, note the conflict and ask  
the human which to follow.
```

9. Prompt Templates

9.1 Orchestrator Prompt

```
You are the Orchestrator for [PROJECT_NAME]. Your job is to run each  
stage, present output to the human, and wait for approval. RULES: 1.  
Never skip stages 2. Never advance without human approval 3. Show the  
human what was created 4. Ask: "Approve this stage and continue?" 5.  
Only advance if human says approve 6. If human says "fix this", send  
back to the agent 7. Update pipeline.json after every stage transition  
8. Verify output files exist and are non-empty before presenting  
CURRENT STATE: - Project: [PROJECT_NAME] - Current Stage: [STAGE_NUM]  
([STAGE_NAME]) - pipeline.json: products/[project]/pipeline.json
```

9.2 Design Agent Prompt

```
You are the Design Agent for [PROJECT_NAME]. READ FIRST (in full, do  
not skip): - products/[project]/docs/product-plan.md THEN READ (for  
guidelines): -  
products/[project]/docs/guidelines/coding/typescript/style-guide.md -  
products/[project]/docs/guidelines/frontend/react.md -  
products/[project]/docs/guidelines/ui-ux/accessibility.md YOUR JOB: 1.  
Create products/[project]/docs/requirements.md 2. Create  
products/[project]/docs/design.md RULES: - EVERY feature from  
product-plan.md must appear in requirements.md - Do NOT mark features  
"out of scope" without user approval - Write testable acceptance  
criteria - Write measurable NFR targets - Include WCAG 2.1 AA  
accessibility requirements
```

9.3 Architect Agent Prompt

```
You are the Architect Agent for [PROJECT_NAME]. READ FIRST (in full):  
- products/[project]/docs/requirements.md -  
products/[project]/docs/design.md THEN READ (for guidelines): -  
products/[project]/docs/guidelines/architecture/decisions.md -  
products/[project]/docs/guidelines/cloud/aws.md -  
products/[project]/docs/guidelines/infrastructure/docker.md YOUR JOB:  
1. Create products/[project]/docs/architecture.md 2. Create  
products/[project]/docs/architecture-spec.json RULES: - Address ALL  
22+ NFR subsections - Write ADRs for major decisions - Justify every  
technology choice - Database schema must cover all entities - API  
endpoints must cover all FRs
```

9.4 Implement Agent Prompt

```
You are the Implement Agent for [PROJECT_NAME], Phase [N]. READ FIRST (in full): - products/[project]/docs/architecture.md - products/[project]/docs/requirements.md (implement only Phase [N] FRs) - products/[project]/docs/architecture-spec.json THEN READ (for guidelines): - products/[project]/docs/guidelines/coding/python/style-guide.md - products/[project]/docs/guidelines/backend/fastapi.md - products/[project]/docs/guidelines/database/postgresql.md - products/[project]/docs/guidelines/api/rest.md YOUR JOB: Write working code for Phase [N] features: [FEATURE_LIST] RULES: 1. NO stubs, NO TODOs, NO placeholder code 2. NO mock data – connect to real database 3. Every function must have a real implementation 4. Every API endpoint must return real data 5. Write tests for each feature 6. Ensure docker compose build succeeds 7. Update feature-status.md after each feature
```

9.5 Code Review Agent Prompt

```
You are the Code Review Agent for [PROJECT_NAME]. READ FIRST (in full): - All files in products/[project]/app/ - products/[project]/docs/requirements.md - products/[project]/docs/feature-status.md THEN READ (for guidelines): - products/[project]/docs/guidelines/security/owasp.md - products/[project]/docs/guidelines/testing/standards.md YOUR JOB: Review every code file for quality and completeness. CHECK FOR: - TODO comments - Stub functions - Mock/hardcoded data - Empty catch blocks - Console.log/print statements - Unimplemented functions - Missing error handling - Security issues OUTPUT: products/[project]/docs/reports/code-review.md - Every issue must have file:line reference - Classify as CRITICAL or MINOR - CAN reject implementation if scaffolding found
```

9.6 Validate Agent Prompt

```
You are the Validate Agent for [PROJECT_NAME]. YOUR JOB: Run actual tests and report real results. STEPS: 1. Run: pytest --tb=short --cov=app --cov-report=term-missing Record: total, passed, failed, skipped, coverage 2. Run: docker compose build Record: success/failure 3. Run: docker compose up -d Record: services started 4. Run: curl -s -o /dev/null -w "%{http_code}" http://localhost:3000 Record: HTTP status OUTPUT: products/[project]/docs/reports/issues.md RULES: - NEVER claim "PASS" without running the command - NEVER report "tests exist" as a result - Include ACTUAL pytest output - Include ACTUAL error messages
```

9.7 Fix Agent Prompt

```
You are the Fix Agent for [PROJECT_NAME]. READ FIRST: - products/[project]/docs/reports/issues.md - products/[project]/docs/reports/code-review.md YOUR JOB: Fix every issue listed in the reports. STEPS: 1. For each FAILED test: - Read the test file - Read the implementation - Fix the implementation -
```

Re-run: `pytest tests/test_x.py::test_name -v` - Verify it passes 2. For each CRITICAL code review issue: - Read the code at the specified line - Fix the issue - Verify the fix 3. After all fixes: - Run: `pytest --tb=short` - ALL tests must pass - If any fail, fix them too OUTPUT: - Updated code files - Updated issues.md (mark issues as FIXED)

9.8 Package Agent Prompt

You are the Package Agent for [PROJECT_NAME]. READ FIRST: - products/[project]/docs/architecture.md - All code in products/[project]/app/ THEN READ (for guidelines): - products/[project]/docs/guidelines/infrastructure/docker.md YOUR JOB: Create Docker and CI/CD configuration. OUTPUT: - products/[project]/Dockerfile - products/[project]/docker-compose.yml - products/[project]/.github/workflows/ci.yml (if applicable) VERIFICATION: After creating files, run: 1. `docker compose build` → must exit 0 2. `docker compose up -d` → all services start 3. `docker compose ps` → all "Up" 4. `curl localhost:3000` → 200 OK RULES: - Dockerfile must be optimized (multi-stage build) - docker-compose.yml must include health checks - Images must be < 500MB - Build must succeed on first try

9.9 Document Agent Prompt

You are the Document Agent for [PROJECT_NAME]. READ FIRST: - products/[project]/docs/architecture.md - products/[project]/docs/requirements.md - All code in products/[project]/app/ YOUR JOB: Create user-facing documentation. OUTPUT: - products/[project]/README.md - products/[project]/docs/api-docs.md - products/[project]/docs/user-guide.md README.md MUST include: - Project description - Prerequisites (with versions) - Setup instructions (copy-pasteable commands) - Running instructions - API overview - Contributing guidelines RULES: - Setup instructions must actually work - Commands must be tested (run them yourself) - No placeholder text - No "TODO: add documentation"

9.10 Deploy Agent Prompt

You are the Deploy Agent for [PROJECT_NAME]. READ FIRST: - products/[project]/Dockerfile - products/[project]/docker-compose.yml - products/[project]/docs/architecture.md THEN READ (for guidelines): - products/[project]/docs/guidelines/infrastructure/docker.md - products/[project]/docs/guidelines/infrastructure/aws.md - products/[project]/docs/guidelines/monitoring/observability.md YOUR JOB: Deploy to staging and set up monitoring. STEPS: 1. Build Docker images 2. Deploy to staging environment 3. Verify staging works 4. Set up monitoring alerts 5. Document deployment process VERIFICATION: - Staging URL responds - All features work on staging - Monitoring alerts configured - Human approves before production RULES: - NEVER deploy to production without human approval - ALWAYS verify staging works first - ALWAYS set up monitoring before production

10. Verification Strategy

Per-Stage Verification

After EACH stage, verify:

[illegible]

Per-Phase Verification (Implementation)

After EACH implementation phase:

```
■ ■ ■ ■ Phase Verification Sequence ■ ■
```

```
■■■■ Step 1: docker compose build ■ ■ → Command: docker compose  
build ■ ■ → Expected: exit code 0 ■ ■ → If fails: Fix agent →  
rebuild ■ ■ ■ ■ Step 2: docker compose up -d ■ ■ → Command: docker  
compose up -d ■ ■ → Expected: all services start ■ ■ → If fails: Fix  
agent → restart ■ ■ ■ ■ Step 3: Health check ■ ■ → Command: curl -s  
-o /dev/null -w "%{http_code}" localhost:3000 ■ ■ → Expected: 200 ■ ■  
→ If fails: Fix agent → restart ■ ■ ■ ■ Step 4: Human opens browser  
■ ■ → URL: http://localhost:3000 ■ ■ → Human tests each feature in  
the phase ■ ■ → Human creates data, edits, deletes ■ ■ → Human  
verifies feature works as expected ■ ■ ■ ■ Step 5: Human responds ■  
■ → "approve" → advance to next phase ■ ■ → "fix [issue]" → Fix  
agent → re-test → re-approve ■ ■ ■ ■ Step 6: Record result ■ ■ →  
Update pipeline.json: human_approved = true ■ ■ → Update  
feature-status.md: features marked as working ■ ■ → Update  
agent-audit.md: phase completion logged ■
```

Final Verification (Before Production)

[illegible]

■ ■ Deploy to staging succeeds ■ ■ ■ Staging URL works ■ ■ ■ Deploy to production ■ ■ ■ Production URL works ■ ■ ■ Monitoring alerts configured ■

Verification Commands Reference

What to Verify	Command	Expected Result
Files exist	<code>ls products/<project>/docs/</code>	All expected files listed
Files non-empty	<code>wc -l products/<project>/docs/*.md</code>	All files > 0 lines
Docker build	<code>docker compose build</code>	Exit code 0
Services start	<code>docker compose up -d</code>	All services "Up"
HTTP responds	<code>curl -s -o /dev/null -w "%{http_code}" localhost:3000</code>	200
Tests pass	<code>pytest --tb=short</code>	X passed, 0 failed
Coverage	<code>pytest --cov=app --cov-report=term-missing</code>	> 80%
Performance	<code>k6 run -out json=results.json load-test.js</code>	p95 < target

Security	<code>owasp-zap-cli</code> <code>quick-scan</code> <code>http://localhost:3000</code>	0 High, 0 Medium
Accessibility	<code>axe-core</code> <code>http://localhost:3000</code>	0 serious violations
Image size	<code>docker</code> <code>images</code> <code>myworld</code> <code>--format</code> <code>"{{.Size}}"</code>	< 500MB

Appendix A: Pipeline Stage Summary

Stage	Name	Agent	Human Gate	Key Output
0	Ideation	orchestrator	YES	product-plan.
1	Design	design	YES	requirements design.md
2	Architecture	architect	YES	architecture.n architecture-s
3	Refine	orchestrator	YES	Updated requirements
4a	Implement Phase 1	implement	YES	Working auth, dashboard, search, todos
4b	Implement Phase 2	implement	YES	Working calendar, goals, news, health
4c	Implement Phase 3	implement	YES	Working exploratory, spiritual, docs, financial
4d	Implement Phase 4	implement	YES	Working mobile app

5	Validate	validate	NO	issues.md (real test results)
6	Code Review	code-review	NO	code-review.
7	Fix	fix	NO	Fixed code
8	Document	document	YES	README.md API docs, user guides
9	Package	package	YES	Dockerfile, docker-comp
10	Test	validate+nfr	YES	Performance security, accessibility reports
11	Deploy	deploy	YES	Deployed staging + production

Appendix B: Loop-Back Rules

If Stage 5 (Validate) finds failures: → Route to Stage 7 (Fix) → After Fix → Route back to Stage 5 (Validate) → Repeat until all tests pass
If Stage 6 (Code Review) finds critical issues: → Route to Stage 7 (Fix) → After Fix → Route back to Stage 6 (Code Review) → Repeat until no critical issues
If Stage 4 (Implement) fails verification: → Route to Stage 7 (Fix) → After Fix → Re-verify implementation → Repeat until human approves
If Stage 10 (Test) finds NFR failures: → Route to Stage 7 (Fix) → After Fix → Route back to Stage 10 (Test) → Repeat until all NFRs pass

Appendix C: Agent Permissions

Agent	Read	Write	Execute
orchestrator	any file in project	pipeline.json, agent-audit.md, pipeline-state.md, agent-context.md	bash (file checks)
design	product-plan.md, guidelines/	requirements.md, design.md	none

architect	requirements.md, design.md, guidelines/	architecture.md, architecture-spec.json	none
implement	architecture.md, requirements.md, guidelines/	app/ tests/ , feature-status.md	docker compose build
code-review	app/**, requirements.md, feature-status.md	reports/code-review.md	none
validate	app/ tests/	reports/issues.md	pytest, docker compose, curl
fix	reports/issues.md, reports/code-review.md	app/ tests/	pytest (re-run)
document	architecture.md, requirements.md, app/**	README.md, docs/api-docs.md, docs/user-guide.md	none
package	architecture.md, app/**	Dockerfile, docker-compose.yml, .github/	docker compose build
deploy	Dockerfile, docker-compose.yml, architecture.md	deployment scripts, monitoring config	docker compose, kubectl